

Manual Técnico

1. Objetivo

El proyecto tiene como finalidad cargar información de sudokus, validarla y generar reportes estadísticos en HTML. La solución está implementada con Python y una estructura MVC simple.

2. Estructura del proyecto

```
Practica/  
├── MCV/  
│   ├── modelo.py  
│   ├── importar.py  
│   ├── validar.py  
│   ├── reportes.py  
│   ├── vista.py  
│   └── main.py  
├── test/  
│   ├── jugadores.lfp  
│   ├── sudokus.lfp  
│   └── intentos.lfp  
├── report/  
│   └── archivos HTML generados  
├── pdf/  
│   ├── ManualUsuario.md  
│   ├── ManualTecnico.md  
│   ├── ManualTecnicoPDF.md  
│   └── DiagramaFlujo.tex  
└── ...
```

3. Descripción de capas

3.1 Modelo

Archivo: Practica/MCV/modelo.py

Contiene las clases:

- Sudoku
- Usuario
- Partida

Cada clase representa una entidad del sistema.

```
class Sudoku:  
    def __init__(self, id_sudoku, dificultad, tablero):  
        self.id_sudoku = id_sudoku
```

```
self.dificultad = dificultad  
self.tablero = tablero
```

3.2 Controlador

Archivo: Practica/MCV/importar.py

Se encarga de:

- leer archivos .lfp
- convertir líneas en objetos
- asociar cada intento con su sudoku correspondiente
- calcular propiedades de validación cuando es necesario

Funciones principales:

- cargar_sudokus(ruta)
- cargar_usuarios(ruta)
- cargar_partidas(ruta, sudokus=None)
- _leer_registros_multilinea(ruta, campos_esperados)

3.3 Validación

Archivo: Practica/MCV/validar.py

Funciones principales:

- cadena_a_matriz(cadena)
- valida_pistas(original, propuesta)
- valida_grupo(grupo)
- valida_tablero(matriz)
- calificar_intento(cadena_original, cadena_solucion)

La validación funciona así:

1. Convierte la cadena en matriz 9x9
2. Verifica que las pistas originales no sean modificadas
3. Comprueba filas, columnas y cajas 3x3
4. Calcula el porcentaje de validez
5. Retorna si la solución está correcta

4. Datos de entrada

Los archivos .lfp contienen registros separados por comas.

Jugadores

Formato:

```
carnet,nombre,apellido,nivel
```

Sudokus

Formato:

```
id_sudoku,dificultad,tablero
```

Intentos

Formato:

```
carnet,id_sudoku,solucion,tiempo_segundos,fecha
```

5. Validación del sudoku

La validación se basa en reglas del sudoku:

- cada fila debe contener 1-9 sin repetir
- cada columna debe contener 1-9 sin repetir
- cada bloque 3x3 debe contener 1-9 sin repetir
- ninguna pista original puede cambiarse

El porcentaje de validez se calcula con la fórmula:

```
porcentaje = (grupos_validos / 27) * 100
```

donde 27 representa:

- 9 filas
- 9 columnas
- 9 cajas

6. Reportes

Archivo: Practica/MCV/reportes.py

Los reportes se generan en HTML y se guardan en la carpeta:

- Practica/report

Reportes principales:

- resumen_sudoku.html

- rendimiento_jugador.html
- top10_tiempos.html

Cada reporte usa tablas HTML con estilo simple para facilitar lectura en navegador.

7. Vista y flujo de menú

Archivo: Practica/MCV/vista.py

La vista se encarga de:

- mostrar menú
- pedir la opción
- leer entradas del usuario
- limpiar pantalla con os.system
- pausar con Enter entre operaciones

El flujo principal está en:

- Practica/MCV/main.py

8. Diagrama de flujo

```
Inicio
↓
Mostrar menú
↓
Leer opción
↓
¿Opción?
├─ 1: Cargar jugadores
├─ 2: Cargar sudokus
├─ 3: Cargar intentos
├─ 4: Validar intento
├─ 5: Reporte sudoku
├─ 6: Reporte jugador
├─ 7: Top 10
└─ 8: Salir

Si opción = 4:
↓
Ingresar tablero original y solución
↓
calificar_intento()
↓
Mostrar resultado
↓
Volver al menú

Si opción = 5, 6 o 7:
↓
Generar archivo HTML
```

```
↓  
Guardar en Practica/report  
↓  
Volver al menú  
  
Si opción = 8:  
↓  
Fin
```

9. Manejo de errores

Se usa try/except para evitar que el programa termine bruscamente si:

- el usuario ingresa menos o más caracteres
- una cadena no tiene 81 dígitos
- el archivo no existe
- el archivo tiene formato incorrecto

Ejemplo:

```
try:  
    resultado = calificar_intento(original, solucion)  
except ValueError as e:  
    print(f"Error en la entrada: {e}")
```

10. Buenas prácticas implementadas

- separación por capas MVC
- uso de clases simples para entidad
- validación aislada en funciones
- generación de reportes en carpeta específica
- mensajes claros en consola
- carga separada por archivo

11. Consideraciones futuras

Se puede mejorar el proyecto con:

- validación más estricta del formato de archivos
- persistencia en base de datos
- interfaz gráfica
- exportación a PDF o Excel
- autenticación de usuarios

12. Conclusión

El proyecto cumple con una arquitectura simple, clara y suficiente para un sistema de análisis básico de sudoku. Su enfoque hace que el código sea fácil de entender, depurar y ampliar.